

UNIT – 3 : OOPS USING JAVA

1. Classes, Objects and Methods

1.1 Java Keywords

Java keywords are reserved words that have predefined meanings in Java programming. These keywords cannot be used as identifiers like variable names, class names, or method names.

Examples: class, public, static, void, int, new, this

1.2 Class

A class is a blueprint or template used to create objects in Java. It defines properties (variables) and behaviors (methods) that objects will have.

```
class Student {  
    int id;  
    String name;  
  
    void display() {  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}
```

1.3 Object

An object is an instance of a class that represents a real-world entity. It is used to access variables and methods defined inside a class.

```
class Test {  
    void show(){  
  
        System.out.println("Hello");  
    }  
    public static void main(String[] args){  
  
        Test obj = new Test();  
  
        obj.show();  
    }  
}
```

Output: Hello

1.4 Types of Variables

Variables are used to store data values. In Java, variables are classified based on scope and usage.

Local Variable: Declared inside a method and accessible only within that method.

```
class Test {  
    void show() {  
        int a = 10;    // local variable  
        System.out.println(a);  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.show();  
    }  
}
```

Output

10

Instance Variable: Declared inside class but outside method. Each object has its own copy.

```
class Student {  
    int id = 101;    // instance variable  
  
    void display() {  
        System.out.println(id);  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

Output

101

Static Variable: It is declared using the `static` keyword. Declared using static keyword and shared among all objects.

```
class Student {  
    static String college = "ABC College";    // static variable  
  
    public static void main(String[] args) {  
        System.out.println(college);  
    }  
}
```

Output

ABC College

1.5 Methods

A method is a block of code that performs a specific task. It helps in code reusability and modular programming.

1.6 Constructors

A constructor is a special method used to initialize objects. It is automatically called when an object is created.

Key Points

- Constructor name must be **same as the class name**
- It does **not have any return type** (not even void)
- It can be **default or parameterized**
- It can be **overloaded** (multiple constructors in one class)

Types of Constructors in Java

1. Default Constructor
2. Parameterized Constructor

1. Default Constructor A default constructor does not take any parameters. It initializes object with default or fixed values.

Code Example

```
class Student {
    int id;

    // Default Constructor
    Student() {
        id = 100;
    }

    void display() {
        System.out.println(id);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Output

100

2. Parameterized Constructor:A parameterized constructor takes arguments. It is used to initialize objects with different values.

Code Example

```
class Student {
    int id;

    // Parameterized Constructor
    Student(int i) {
        id = i;
    }

    void display() {
        System.out.println(id);
    }

    public static void main(String[] args) {
        Student s = new Student(200);
        s.display();
    }
}
```

Output

200

Code (When variable name & parameter name are same in parameterized constructor)

```
class Student {
    int id;

    Student(int id) {
        id = id; // problem: both refer to parameter
    }

    void display() {
        System.out.println(id);
    }

    public static void main(String[] args) {
        Student s = new Student(100);
        s.display();
    }
}
```

Output

0

Explanation

- `id = id` → both refer to **parameter**
- Instance variable `id` remains **default value (0)**

To resolve it, we use this keyword.

1.7 this Keyword

The `this` keyword refers to the current object of the class. It is used to differentiate instance variables from parameters.

```
class Student {
    int id;

    Student(int id) {
        this.id = id; // correct assignment
    }

    void display() {
        System.out.println(id);
    }

    public static void main(String[] args) {
        Student s = new Student(100);
        s.display();
    }
}
```

Output

100

1.8 Static Members

Static members belong to the class rather than objects. They can be accessed without creating an object.

Static Method

A static method is a method that belongs to the class rather than objects. It can be called directly using the class name without creating an object.

Code Example

```
class Test {
```

```
// static method
static void show() {
    System.out.println("This is static method");
}

public static void main(String[] args) {
    Test.show(); // calling without object
}
}
```

Output

This is static method

2. OOPS Concepts

OOPS is a programming approach based on objects and classes. It improves code reusability and maintainability.

It improves code reusability, maintainability, and helps in designing real-world applications.

Pillars of OOPS

1. **Inheritance**
2. **Encapsulation**
3. **Polymorphism**
4. **Abstraction**

1. Inheritance

Types of Inheritance

1. Single Inheritance-Single inheritance occurs when one class inherits from one parent class. It is the simplest form of inheritance and promotes code reuse.

Diagram

A
↓
B

Code

```
class A {  
    void show() {  
        System.out.println("Parent Class");  
    }  
}  
  
class B extends A {  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.show();  
    }  
}
```

Output

Parent Class

2. Multilevel Inheritance-Multilevel inheritance occurs when a class is derived from another derived class.It forms a chain of inheritance.

Diagram

$A \rightarrow B \rightarrow C$

Code

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}  
  
class B extends A {  
}  
  
class C extends B {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Output

Class A

3. Hierarchical Inheritance-Hierarchical inheritance occurs when multiple classes inherit from a single parent class.

Each child class gets access to parent properties.

Diagram



Code

```
class A {  
    void show() {  
        System.out.println("Parent Class");  
    }  
}
```

```
class B extends A {  
}
```

```
class C extends A {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Output

Parent Class

4. Multiple Inheritance -Multiple inheritance occurs when a class inherits from more than one parent class.

Java does not support it using classes to avoid ambiguity (diamond problem).

Diagram



Code (Will Give Error)

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}
```

```
class B {  
    void show() {  
        System.out.println("Class B");  
    }  
}
```



```
}  
}  
  
// ERROR: Java does not support multiple inheritance with classes  
class C extends A, B {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Output (Compile-Time Error)

Error: class C cannot extend multiple classes

To resolve multiple inheritance problem in Java, we use interfaces.

Interface:An interface is a collection of abstract methods that a class must implement. It is used to achieve multiple inheritance and abstraction in Java.

Key Points

- Methods are **public and abstract by default**
 - Variables are **public, static, final**
 - A class uses implements keyword
 - Multiple interfaces can be implemented
-

Syntax

```
interface A {  
    void show();  
}
```

Multiple Inheritance Using Interface (Correct Way)

Code

```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}
```

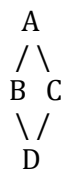
```
}  
  
class C implements A, B {  
  
    public void show() {  
        System.out.println("From A");  
    }  
  
    public void display() {  
        System.out.println("From B");  
    }  
  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
        obj.display();  
    }  
}
```

Output

From A
From B

5. Hybrid Inheritance: Hybrid inheritance is a combination of two or more types of inheritance (like hierarchical + multiple).
Java does not support hybrid inheritance using classes, but it can be achieved using interfaces.

Diagram



Here:

- A → Parent
 - B & C → Derived (Hierarchical)
 - D → Multiple inheritance using interfaces
-

Code Example (Using Interface)

```
interface A {  
    void showA();  
}  
  
interface B extends A {  
    void showB();  
}  
  
interface C extends A {  
    void showC();  
}  
  
class D implements B, C {  
  
    public void showA() {  
        System.out.println("From A");  
    }  
  
    public void showB() {  
        System.out.println("From B");  
    }  
  
    public void showC() {  
        System.out.println("From C");  
    }  
  
    public static void main(String[] args) {  
        D obj = new D();  
        obj.showA();  
        obj.showB();  
        obj.showC();  
    }  
}
```

Output

From A
From B
From C

Access Modifiers in Java

Access modifiers are keywords used to define the accessibility of classes, variables, and methods.

They control where and how members of a class can be accessed in a program.

Types of Access Modifiers

1. Public
 2. Private
 3. Protected
 4. Default (No Modifier)
-

1. Public: Public members are accessible from anywhere in the program. They can be accessed from any class, even outside the package.

Syntax

```
public int a;
```

Code Example

```
class Test {  
    public int a = 10;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Test obj = new Test();  
        System.out.println(obj.a);  
    }  
}
```

Output

```
10
```

2. Private: Private members are accessible only within the same class. They cannot be accessed outside the class directly.

Syntax

```
private int a;
```

Code Example

```
class Test {  
    private int a = 10;  
  
    void show() {  
        System.out.println(a);  
    }  
}
```

```

}

class Main {
    public static void main(String[] args) {
        Test obj = new Test();
        obj.show(); // accessing via method
    }
}

```

Output

10

3. Protected: Protected members are accessible within the same package and in subclasses. They support inheritance by allowing child classes to access parent members.

Syntax

protected int a;

Code Example

```

class A {
    protected int a = 10;
}

class B extends A {
    void show() {
        System.out.println(a);
    }

    public static void main(String[] args) {
        B obj = new B();
        obj.show();
    }
}

```

Output

10

4. Default (No Modifier): Default members are accessible only within the same package. They cannot be accessed outside the package.

Syntax

int a;

Code Example

```

class Test {
    int a = 10; // default access
}

```

```
}  
  
class Main {  
    public static void main(String[] args) {  
        Test obj = new Test();  
        System.out.println(obj.a);  
    }  
}
```

Output

10

2. Encapsulation- Encapsulation is the process of wrapping data (variables) and code (methods) into a single unit. It is used to protect data by making variables private and accessing them through methods.

Key Idea

Data hiding using **private variables + getter/setter methods**

Code Example

```
class Student {  
    private int id; // private variable  
  
    void setId(int i) {  
        id = i;  
    }  
  
    int getId() {  
        return id;  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.setId(101);  
        System.out.println(s.getId());  
    }  
}
```

Output

101

3. Polymorphism- Polymorphism means "many forms", where one method behaves differently in different situations.

It allows the same function name to perform different tasks.

Types of Polymorphism

1. Compile-time (Method Overloading)
 2. Runtime (Method Overriding)
-

(A) Method Overloading (Compile-Time) - Method overloading is a feature where multiple methods have the same name but different parameters.

It is achieved within the same class and is an example of compile-time polymorphism.

Code Example

```
class Test {  
    void add(int a, int b) {  
        System.out.println(a + b);  
    }  
  
    void add(int a, int b, int c) {  
        System.out.println(a + b + c);  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.add(2, 3);  
        obj.add(2, 3, 4);  
    }  
}
```

Output

5
9

(B) Method Overriding (Runtime) - Method overriding is a feature where a child class provides a specific implementation of a method already defined in the parent class.

It is achieved using inheritance and is an example of runtime polymorphism.

Code Example

```
class A {  
    void show() {  
        System.out.println("Parent Class");  
    }  
}
```

```
class B extends A {  
    void show() {  
        System.out.println("Child Class");  
    }  
  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.show();  
    }  
}
```

Output

Child Class

To call parent class function also , we use super keyword.

Definition of super keyword (Java)

super is a reference variable used to refer to the immediate parent class object.

It is mainly used to:

- Access **parent class methods**
 - Access **parent class variables**
 - Call **parent class constructor**
-

Syntax of super

1Call Parent Class Method

```
super.methodName();
```

2Access Parent Class Variable

```
super.variableName;
```

3Call Parent Class Constructor

```
super();    // default constructor  
super(arguments); // parameterized constructor
```

Example (Method Call)

```
class Parent {  
    void show() {  
        System.out.println("Parent method");  
    }  
}
```



```
}  
  
class Child extends Parent {  
    void show() {  
        super.show(); // calling parent method  
    }  
}
```

4. Abstraction- Abstraction means hiding implementation details and showing only essential features.

It is used to reduce complexity and increase efficiency.

Ways to Achieve Abstraction

1. Abstract Class - An abstract class is a class that cannot be instantiated (object cannot be created).
It can contain abstract methods (without body) as well as normal methods (with body).
 2. Interface
-

Using Abstract Class

Code Example

```
abstract class A {  
    abstract void show();  
}  
  
class B extends A {  
    void show() {  
        System.out.println("Abstract Method");  
    }  
  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.show();  
    }  
}
```

Output - Abstract Method

3. Arrays, Strings and Vectors

1. Array (1D Array) - An array is a collection of similar data types stored in contiguous memory locations.

A 1D array stores elements in a single row using one index.

Syntax

```
int arr[] = new int[5];
```

Code Example

```
class Test {  
    public static void main(String[] args) {  
        int arr[] = {10, 20, 30};  
  
        for(int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

Output

```
10  
20  
30
```

2. Creating an Array - Array creation means allocating memory to store elements. It is done using the new keyword.

Code Example

```
class Test {  
    public static void main(String[] args) {  
        int arr[] = new int[3];  
        arr[0] = 5;  
        arr[1] = 10;  
        arr[2] = 15;  
  
        System.out.println(arr[1]);  
    }  
}
```

Output

```
10
```

3. 2D Array - A 2D array is an array of arrays used to store data in rows and columns. It is also called a matrix.

Syntax

```
int arr[][] = new int[2][2];
```

Code Example

```
class Test {  
    public static void main(String[] args) {  
        int arr[][] = {{1,2},{3,4}};  
  
        for(int i=0;i<2;i++){  
            for(int j=0;j<2;j++){  
                System.out.print(arr[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
1 2  
3 4
```

4. **Strings** - String is a sequence of characters used to store text. In Java, strings are objects of the String class.

Code Example

```
class Test {  
    public static void main(String[] args) {  
        String name = "Java";  
        System.out.println(name);  
    }  
}
```

Output

```
Java
```

5. Vectors - Vector is a dynamic array that can grow or shrink in size automatically. It is part of Java's collection framework.

Code Example

```
import java.util.Vector;

class Test {
    public static void main(String[] args) {
        Vector v = new Vector();

        v.add(10);
        v.add(20);

        System.out.println(v);
    }
}
```

Output

[10, 20]

6. Wrapper Classes - Wrapper classes convert primitive data types into objects. They are used in collections where objects are required.

Examples

- int → Integer
 - char → Character
-

Code Example

```
class Test {
    public static void main(String[] args) {
        int a = 10;
        Integer obj = a; // autoboxing

        System.out.println(obj);
    }
}
```

Output

10

7. Enumerated Types (Enum) - Enum is a special data type used to define a group of constants.

It makes code more readable and manageable.

Code Example

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY  
}  
  
class Test {  
    public static void main(String[] args) {  
        Day d = Day.MONDAY;  
        System.out.println(d);  
    }  
}
```

Output

MONDAY